

HOTEL BOOKING MANAGEMENT SYSTEM

Python Command-Line Application

PROJECT REPORT

Developed as part of the Python Essentials / FlipCourse Project

Technology: Python 3

Application Type: Command-Line Interface

Data Storage: CSV Files

Final Project Submission Report

1. Introduction

The Hotel Booking Management System is a modular command-line Python application designed to support the basic operations of a small hotel. The system combines room management, customer management, booking management, cancellation, billing, and reporting into one application.

The project demonstrates foundational Python programming concepts including functions, modules, conditional statements, loops, input validation, file handling, CSV processing, and modular program design.

Persistent information is maintained using CSV files so that room, customer, and booking information remains available after the application is closed.

2. Problem Statement

Managing hotel rooms, customer information, bookings, and billing manually can be time-consuming and may lead to errors. A simple computerized system can organize these activities and provide a consistent workflow for hotel staff.

The proposed system provides a centralized command-line solution for maintaining hotel room records, customer records, booking records, billing calculations, and basic operational reports.

3. Objectives

- Develop a functional hotel booking management application using Python.
- Provide separate modules for room, customer, booking, billing, reporting, validation, and storage operations.
- Allow hotel staff to perform basic CRUD operations on room and customer records.
- Prevent invalid booking operations such as booking an occupied or nonexistent room.
- Automatically calculate bills using room price and number of nights.
- Maintain persistent data using CSV files.
- Demonstrate modular programming and maintainable project organization.
- Use Git and GitHub for version control and project delivery.

4. Scope of the Project

- Room viewing, addition, updating, and deletion of available rooms.
- Customer addition, viewing, updating, and deletion.
- Booking creation, viewing, and cancellation.
- Automatic room-status updates during booking and cancellation.
- Bill generation for active bookings.
- Hotel summary and available-room reports.
- CSV-based local data persistence.
- Command-line interaction.

Out of scope: online payments, customer login, online reservation portals, real-time multi-user synchronization, and external database servers.

5. Functional Requirements

Module	Functional Requirements
Room Management	View rooms; add rooms; update room details; delete available rooms; track room status.
Customer Management	Add, view, update, and delete customer records.
Booking Management	Create bookings; verify customer; verify room; check availability; view bookings; cancel bookings.
Billing	Generate a bill for an active booking using room price and number of nights.
Reports	Display total rooms, available rooms, occupied rooms, customers, active bookings, cancellations.

6. Non-Functional Requirements

Requirement	Description
Usability	The system should be simple and easy to operate through a menu-driven command-line interface.
Reliability	The system validates user input and prevents invalid operations such as booking an occupied room.
Maintainability	The application is divided into separate Python modules for different functionalities.
Data Persistence	Room, customer, and booking information remains stored in CSV files after the application terminates.
Performance	Basic operations should execute quickly for a small hotel dataset.
Error Handling	The system displays meaningful messages when invalid input or unavailable records are encountered.

7. Target Users

- Hotel reception staff
- Small hotel administrators
- Hotel booking operators

8. Technology Stack

Technology / Tool	Purpose
Python 3	Application development
CSV Module	Reading and writing persistent CSV data
Command Line Interface	User interaction
Git	Version control
GitHub	Remote repository and project hosting

9. System Architecture

The system follows a modular architecture. The command-line interface is controlled by main.py, which routes user choices to specialized modules.

Validation is handled by validation.py, while CSV file operations are handled by storage.py. The main functional modules are room_manager.py, customer_manager.py, booking_manager.py, billing.py, and reports.py.

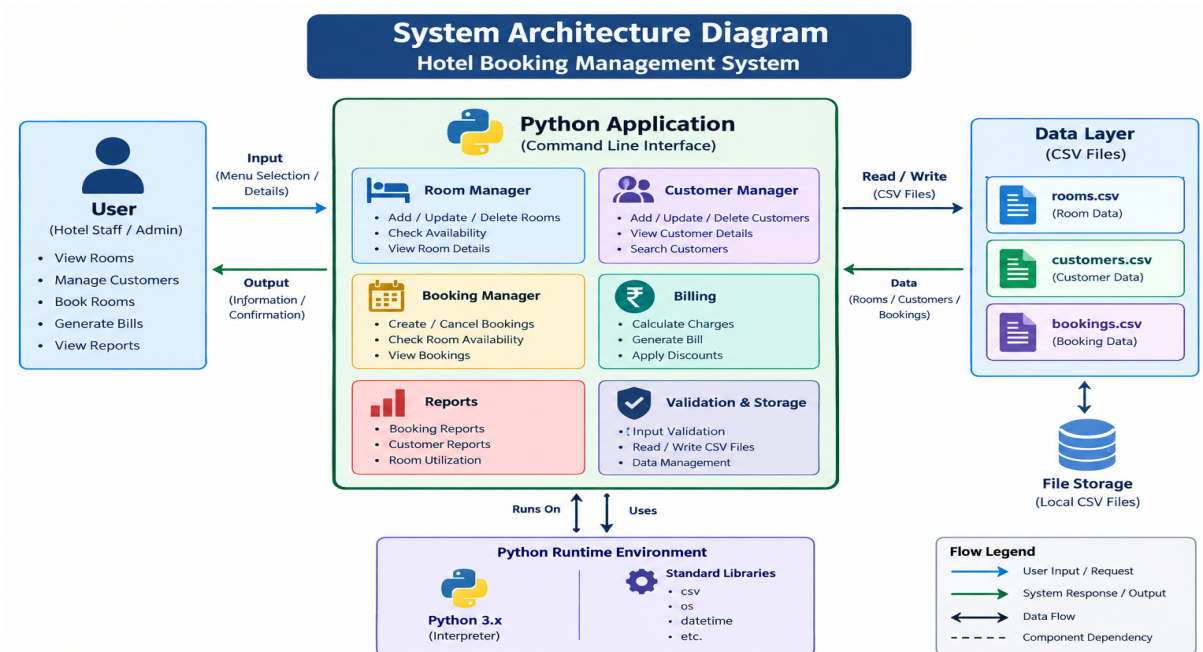


Figure 1. System Architecture

10. Project Structure

Path	Purpose
src/main.py	Main menu and application control
src/room_manager.py	Room operations
src/customer_manager.py	Customer operations
src/booking_manager.py	Booking and cancellation operations
src/billing.py	Bill calculation and display
src/reports.py	Hotel reports
src/validation.py	Input validation
src/storage.py	CSV reading and writing
data/*.csv	Persistent room, customer, and booking records
tests/	Automated validation tests
docs/	Requirements, design documentation, and diagrams

11. Use Case Design

The primary actor is the Hotel Staff member. The staff interacts with the command-line system to manage rooms and customers, create or cancel bookings, generate bills, and view reports.

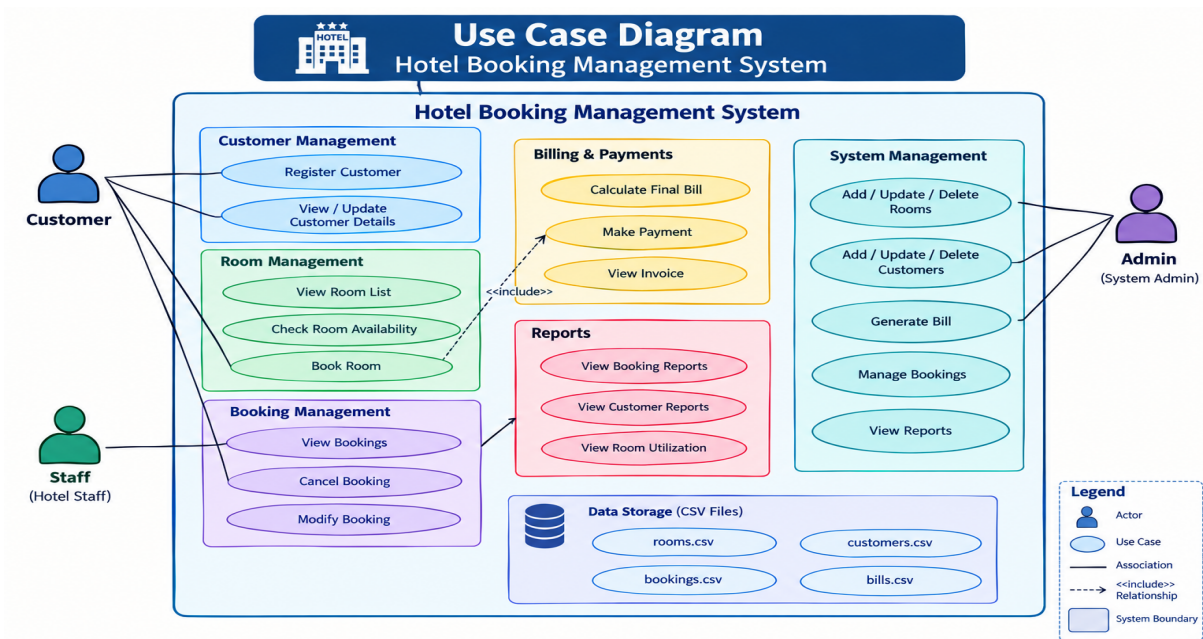


Figure 2. Use Case Diagram

12. System Workflow

A typical booking workflow begins when hotel staff selects Booking Management. The system verifies the customer, verifies the room, checks room availability, records the booking, updates the room status, and allows billing and reporting operations.

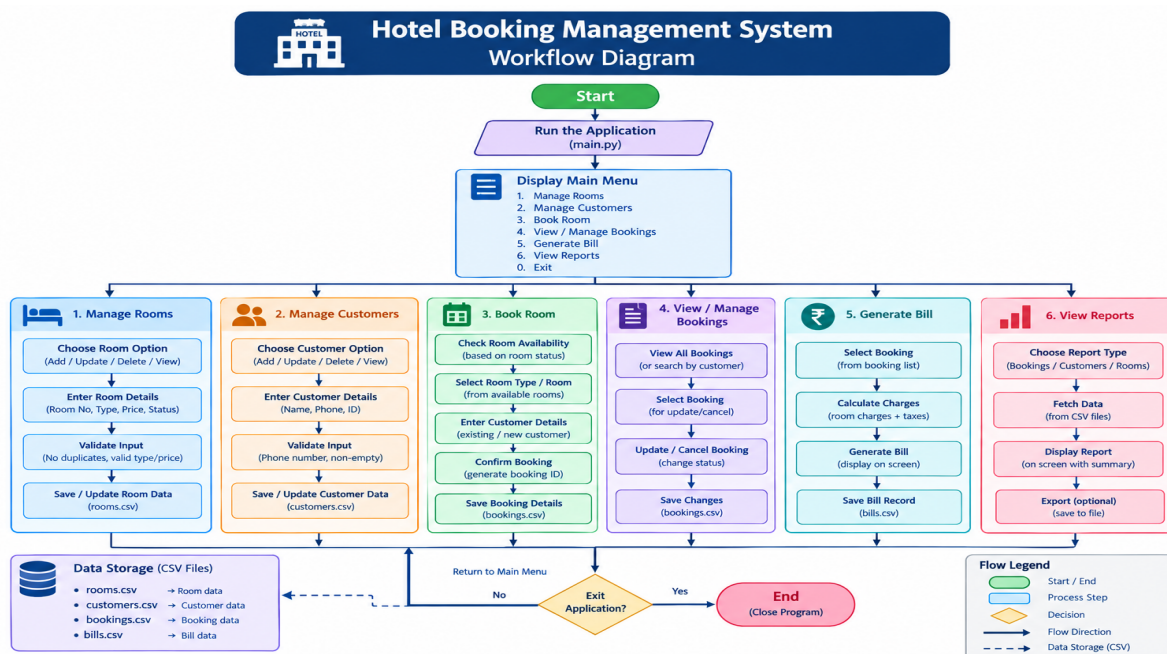


Figure 3. System Workflow

13. Class / Component Design

The application is organized into separate Python components. `main.py` controls the application flow, while `room_manager.py`, `customer_manager.py`, and `booking_manager.py` implement the main operational modules. `billing.py` and `reports.py` provide supporting business functions.

Component	Responsibility
<code>main.py</code>	Main menu and application control
<code>room_manager.py</code>	Room management
<code>customer_manager.py</code>	Customer management
<code>booking_manager.py</code>	Booking and cancellation
<code>billing.py</code>	Bill generation
<code>reports.py</code>	Hotel reports
<code>validation.py</code>	Input validation
<code>storage.py</code>	CSV data storage

14. Sequence Design

1. The user selects Booking Management from the main menu.
2. `main.py` calls the booking management function.
3. `booking_manager.py` requests the customer ID.
4. The system checks customer information using `storage.py`.
5. The user enters the room number.
6. The system checks room information and availability.
7. The user enters the number of nights.
8. The booking is stored in `bookings.csv`.
9. The room status is changed to Occupied.
10. The updated room information is stored in `rooms.csv`.
11. The system displays a successful booking message.

15. Data Storage and ER Design

The system uses three CSV files for persistent storage. Customer and room identifiers are referenced by booking records to connect related information.

File	Fields
<code>rooms.csv</code>	<code>room_number</code> , <code>room_type</code> , <code>price</code> , <code>status</code>
<code>customers.csv</code>	<code>customer_id</code> , <code>name</code> , <code>phone</code> , <code>email</code>
<code>bookings.csv</code>	<code>booking_id</code> , <code>customer_id</code> , <code>room_number</code> , <code>number_of_nights</code> , <code>status</code>

Relationships: One customer can have multiple bookings. Each booking is associated with one customer and one room. Room status is updated when a booking is created or cancelled.

16. Design Decisions and Rationale

Decision	Rationale
Modular Python files	Separates responsibilities and makes the project easier to understand and maintain.
CSV storage	Simple, lightweight, human-readable storage suitable for a small academic CLI application.
Central validation module	Avoids repeating common input-validation logic.
Menu-driven CLI	Provides a straightforward interface using core Python concepts.
Room status tracking	Prevents double booking and releases rooms after cancellation.

17. Implementation Details

Room Management: The system reads rooms from rooms.csv and supports viewing, adding, updating, and deleting rooms. Occupied rooms cannot be deleted.

Customer Management: Customer records are stored in customers.csv. Customer IDs are checked for duplicates before a new customer is added.

Booking Management: A booking can only be created when the customer exists and the selected room exists and is available. A successful booking changes the room status to Occupied.

Cancellation: Cancelling an active booking changes its status to Cancelled and changes the related room status back to Available.

Billing: The total amount is calculated by multiplying the room price per night by the number of nights.

Reports: The reports module counts rooms, customers, and bookings and provides a list of currently available rooms.

Validation: validation.py handles non-empty input, positive numbers, and positive integers.

18. Testing Approach

The project uses both manual application testing and automated unit testing for the validation module. The automated tests use Python's built-in unittest framework.

Test Case	Expected Result	Result
Launch application	Main menu displayed	Passed
View rooms	Room list displayed	Passed
Add customer	Customer record saved	Passed
Create booking	Booking created and room becomes Occupied	Passed
Generate bill	Correct total calculated	Passed
Hotel summary	Hotel statistics displayed	Passed
Cancel booking	Booking cancelled and room becomes Available	Passed
Invalid numeric input	Validation message displayed	Implemented
Book occupied room	Booking rejected	Implemented

Automated validation testing: 3 tests were executed using unittest and all 3 tests passed successfully.

The three automated tests cover non-empty input, positive number input, and positive integer input.

A representative manual test used customer ID C001, room 101, and two nights. Room 101 costs ■1500 per night, resulting in a calculated bill of ■3000.

19. Results

- The application successfully launches from the command line.
- Room records can be viewed and maintained.
- Customer records can be added and maintained.
- Bookings update room occupancy status.
- Cancelled bookings release the associated room.
- Billing calculates the total from room price and number of nights.
- Reports summarize current hotel data.
- CSV files provide persistent local storage.
- Automated validation tests execute successfully.

20. Challenges

- Designing separate modules while keeping the application workflow simple.
- Keeping room and booking statuses synchronized across CSV files.
- Handling invalid input without terminating the program.
- Maintaining consistent field names across multiple CSV files.
- Organizing source code, documentation, diagrams, tests, and GitHub files.

21. Learnings

- Practical use of Python modules and functions.
- Reading and writing structured data using CSV files.
- Using loops and conditions to implement application logic.
- Implementing validation and error handling.
- Designing a modular command-line application.
- Using unittest for automated testing.
- Using Git for version control and GitHub for project hosting.
- Preparing technical documentation and software design diagrams.

22. Future Enhancements

- Replace CSV storage with SQLite or MySQL.
- Add check-in and check-out dates.

- Add customer search and filtering.
- Add automated invoice file generation.
- Add authentication and role-based access.
- Add a graphical or web-based user interface.
- Add more automated unit and integration tests.
- Add occupancy and revenue analytics.

23. GitHub and Version Control

The project is maintained in a Git repository with the main branch hosted on GitHub. The repository contains the source code, CSV data files, documentation, diagrams, README.md, statement.md, requirements.txt, tests, and project configuration.

GitHub Repository: github.com/PRANJALBHATI/Hotel-Booking-Management-System

Git commits were used to track the initial implementation, generated-file cleanup, documentation, design diagrams, and automated testing updates.

24. Conclusion

The Hotel Booking Management System provides a complete small-scale hotel management workflow using Python and CSV storage. It demonstrates core programming concepts together with modular software design, input validation, persistent data handling, documentation, testing, and Git-based version control.

The project provides a foundation that can be extended with database storage, dates, authentication, automated testing, and a graphical or web interface in future versions.

25. References

- Python 3 Standard Library documentation and Python programming concepts.
- Git documentation and GitHub version-control workflow.
- Python Essentials / FlipCourse project requirements and specifications.
- Project source code and documentation contained in the Hotel Booking Management System repository.